

There is a core function for a meeting management system designed for government and enterprise clients. The algorithm calculates the video resolution and total server bandwidth allocation based on the participant count and the client's subscription tier.

We are using Clean Architecture to separate the core business rules (Domain) and the application logic (Use Case) from the UI.

meeting_types:

```
export type MeetingTier = 'STANDARD' | 'GOVERNMENT';
export type Resolution = '1080p' | '720p' | '480p' | 'AUDIO_ONLY';

export interface AllocationResult {
  resolution: Resolution;
  bandwidthMbps: number;
}
```

calculate-allocation:

```
import type {
  MeetingTier,
  AllocationResult
} from '../domain/meeting-types';

export const calculateAllocation = (
  participants: number,
  tier: MeetingTier
): AllocationResult => {
  if (!Number.isInteger(participants) || participants < 1) {
    throw new Error('Participants must be a positive integer.');
```

```
  }

  if (tier === 'GOVERNMENT') {
    if (participants > 100) {
      throw new Error('Government nodes support a maximum of 100 users.');
```

```
    }
    if (participants <= 20) {
      // 4 Mbps per user for 1080p
      return { resolution: '1080p', bandwidthMbps: participants * 4 };
    }
    // 2 Mbps per user for 720p
    return { resolution: '720p', bandwidthMbps: participants * 2 };
  }

  // STANDARD tier logic
  if (participants > 100) {
    throw new Error('Standard tier supports a maximum of 100 users.');
```

```
  }
  if (participants <= 10) {
    // 2 Mbps per user for 720p
    return { resolution: '720p', bandwidthMbps: participants * 2 };
  }
}
```

```

if (participants <= 50) {
  // 1 Mbps per user for 480p
  return { resolution: '480p', bandwidthMbps: participants * 1 };
}
// 0.1 Mbps per user for Audio Only
return { resolution: 'AUDIO_ONLY', bandwidthMbps: participants * 0.1 };
};

```

There is a React component that acts as the controller and presenter, it is called: AllocationCalculator

```

import React, { useState } from 'react';
import type { FormEvent } from 'react';
import type { MeetingTier, AllocationResult } from '../domain/meeting-types';
import { calculateAllocation } from '../use-cases/calculate-allocation';

export const AllocationCalculator: React.FC = () => {
  const [participants, setParticipants] = useState<string>("");
  const [tier, setTier] = useState<MeetingTier>('STANDARD');
  const [result, setResult] = useState<AllocationResult | null>(null);
  const [error, setError] = useState<string | null>(null);

  const handleSubmit = (e: FormEvent<HTMLFormElement>): void => {
    e.preventDefault();
    setError(null);
    setResult(null);

    try {
      const parsedCount = Number(participants);
      const allocation = calculateAllocation(parsedCount, tier);
      setResult(allocation);
    } catch (err: unknown) {
      if (err instanceof Error) {
        setError(err.message);
      } else {
        setError('An unexpected error occurred.');
```

```

      onChange={(e) => setParticipants(e.target.value)}
      required
    />
  </div>

  <div>
    <label htmlFor="tier">Meeting Tier:</label>
    <select
      id="tier"
      value={tier}
      onChange={(e) => setTier(e.target.value as MeetingTier)}
    >
      <option value="STANDARD">Standard</option>
      <option value="GOVERNMENT">Government</option>
    </select>
  </div>

  <button type="submit">Calculate Allocation</button>
</form>

{error && <div className="error-box">{error}</div>}

{result && (
  <div className="result-box">
    <p><strong>Resolution:</strong> {result.resolution}</p>
    <p><strong>Total Bandwidth:</strong> {result.bandwidthMbps} Mbps</p>
  </div>
)}
</div>
);
};

```

Building the project:

- Step 1: install node & npm
- Step 2: open vscode
- Step 3: bootstrap a new React TypeScript project:
npm create vite@latest bandwidth-calculator -- --template react-ts
- Step 4: go to bandwidth-calculator folder & type npm init
- Step 5:
In src/domain/ create meeting-types.ts and paste the domain types.
In src/use-cases/ create calculate-allocation.ts and paste the business logic.
In src/presentation/ create AllocationCalculator.tsx and paste the UI component code.
- Step 6: Open the existing src/App.tsx file and replace the content with the followings

```

import { AllocationCalculator } from './presentation/AllocationCalculator';
import './App.css'; // Optional: Keeps default styling, or you can remove this

function App() {
  return (
    <div className="App">

```

```
    <AllocationCalculator />
  </div>
);
}

export default App;
```

- Step 7: type “npm run dev”

Request:

1. Run the project and capture the screen of app; then paste it to the sheet “functions” of file [Class]_[Student ID]_[Student Fullname]_UnitTest.xlsx
2. Create unit test case for above code, and fill them to the file [Class]_[Student ID]_[Student Fullname]_UnitTest.xlsx
3. Update filename using your own information and upload to LMS (Lab 1)